Roshan

Unified Observability Platform

Metrics | Traces | Logs

Official Documentation, Version 4.2.0 (Sunrise)

Synthetic documentation. Roshan, Roshan Labs and all figures are fictional.
Created for testing documentation-aware AI agents. Published November 2026.

Contents

1. What is the Roshan Platform? 3

2. Why Roshan Is the Best Observability Platform 5

3. Architecture 7

4. Installation and Getting Started 9

5. Metrics 11

6. Traces 15

7. Logs 18

8. Correlation: One Click Across Signals 21

9. Alerting with Roshan Sentinel 22

10. Dashboards and the Roshan Console 24

11. RQL: Roshan Query Language 25

12. Comparisons with Other Platforms 27

13. Security, Access Control and Multi-Tenancy 30

14. Deployment, Sizing and Operations 31

15. Configuration Reference 33

16. API and CLI Reference 35

17. Migration Guides 37

18. Pricing and Licensing (Fictional) 38

19. Troubleshooting 39

20. Frequently Asked Questions 40

21. Glossary 44

22. Release Notes 45

23. Quick Reference Card 46

1. What is the Roshan Platform?

> **Notice:** Roshan is a fictional product. This document is synthetic reference material written for testing documentation-aware AI agents. All product names, versions, benchmarks, prices, flags and APIs are invented and must not be treated as factual claims about any real system.

1.1 Overview

Roshan is a unified, open-core observability platform that collects, stores, queries, correlates and alerts on the three core telemetry signals: **metrics**, **traces** and **logs**. It was created by the fictional company Roshan Labs and first released in March 2022. The current stable release described in this documentation is **Roshan 4.2 (codename Sunrise)**.

Unlike stacks that stitch together several independent products, Roshan ships as a single platform with one agent, one gateway, one query language (RQL), one access-control model and one console. Internally each signal has a purpose-built storage engine, but the user experiences them as a single system.

The most distinctive design decision in Roshan is its metrics engine. Instead of the classic Prometheus TSDB, Roshan's metrics store (**Roshan Store-M**) is built on **VictoriaMetrics** technology, extended with Roshan's own sharding layer, tenant-aware compaction, adaptive downsampling and an exemplar index. This gives Roshan very high ingestion rates, excellent compression and long-term retention without the need for a separate long-term storage project.

1.2 Product components

Component	Binary	Role
Roshan Agent	roshan-agent	Node/pod-level collector for metrics, logs, traces and profiles. Supports scraping, OTLP receiving, file tailing and eBPF auto-instrumentation.
Roshan Gateway	roshan-gateway	Stateless ingest tier. Authenticates, rate-limits, validates, routes and batches telemetry to the stores.
Roshan Store-M	roshan-store-m	Metrics storage built on VictoriaMetrics, with tenant-aware sharding and adaptive downsampling.
Roshan Store-T	roshan-store-t	Trace storage. Columnar parquet-like blocks on object storage with a bloom-filter and attribute index.
Roshan LogLake	roshan-loglake	Log storage built on a VictoriaLogs-derived engine with token-bloom indexing and columnar compression.
Roshan Query Engine	roshan-query	Executes RQL queries across all stores, with caching, fan-out and cross-signal joins.
Roshan Sentinel	roshan-sentinel	Alerting, routing, silencing and incident correlation.
Roshan Correlator	roshan-correlator	Builds the entity graph and links metrics, traces and logs using trace IDs, exemplars and resource attributes.
Roshan Pipelines	roshan-pipelines	Transformation, enrichment, redaction and sampling rules applied in-flight.
Roshan Console	roshan-console	Web UI: explore, dashboards, service maps, alerts, admin and billing views.

1.3 Key capabilities at a glance

- Single agent and single protocol surface: OTLP (gRPC/HTTP), Prometheus remote write, Prometheus scrape, StatsD, Syslog, Loki push compatibility, Jaeger and Zipkin.
- **RQL** (Roshan Query Language): a superset of PromQL and MetricsQL, with log and trace extensions and cross-signal joins.

- Metrics: up to **18 million samples/second per Store-M cluster node group** (fictional reference figure), around 0.4 bytes per sample after compression.
- Traces: 100% ingestion with tail-based sampling at the gateway; cost-based retention.
- Logs: schema-on-read with automatic field extraction; typical 12x to 20x compression.
- Built-in correlation: click from an alerting metric to an exemplar trace to the exact log lines.
- Native multi-tenancy, RBAC, SSO (OIDC, SAML), audit logging and field-level redaction.
- Deployable as SaaS (Roshan Cloud), self-hosted Kubernetes (Helm/Operator) or a single binary for edge and development use.

1.4 Who uses Roshan?

The fictional reference customer base includes platform engineering teams, SREs, DevOps engineers, security operations teams and FinOps analysts. Roshan targets organizations running between 20 and 20,000 services, though the single-binary edition runs comfortably on a laptop for development and testing.

1.5 Editions

Edition	Hosting	Highlights	Limits
Roshan Community	Self-hosted	All three signals, RQL, single tenant, basic alerting, community support.	One tenant, 30-day max retention, no SSO.
Roshan Team	Self-hosted or Cloud	Multi-tenant, RBAC, SSO, Sentinel routing, email support.	Up to 50 tenants, 13-month retention.
Roshan Enterprise	Self-hosted or Cloud	Unlimited tenants, audit logs, field redaction, cross-region replication, 24x7 support, custom retention.	None enforced by software.
Roshan Edge	Self-hosted	Single binary, SQLite-style metadata, ideal for IoT and branch sites.	Max 50k active series, 7-day retention.

1.6 Release history

Version	Codename	Date	Major features
1.0	Dawn	March 2022	Metrics on Store-M, basic Console, Prometheus remote write.
2.0	Daybreak	February 2023	Traces (Store-T), OTLP-native ingest, service maps.
3.0	Meridian	January 2024	LogLake, RQL 1.0, Correlator, Sentinel alerting.
3.5	Zenith	November 2024	Adaptive downsampling, tail sampling at gateway, eBPF agent.
4.0	Solstice	September 2025	Cross-signal joins, cost governance, profiles preview.
4.2	Sunrise	October 2026	Anomaly-aware alerts, cardinality shield 2.0, RQL 2.2.

2. Why Roshan Is the Best Observability Platform

This chapter explains, in the fictional context of this documentation, why Roshan describes itself as the best platform in the observability world. Each claim is tied to a specific engineering decision. Detailed side-by-side tables appear in Chapter 12.

2.1 The nine pillars of the Roshan advantage

Pillar 1 - VictoriaMetrics-based metrics engine instead of the Prometheus TSDB

Prometheus stores data in a local, single-node TSDB made of two-hour blocks. It is excellent for small and medium installations but needs federation, Thanos, Cortex or Mimir to scale horizontally and retain data long term. Roshan Store-M is built on VictoriaMetrics, which uses a MergeTree-style storage design with strong compression and low memory use. In Roshan's reference benchmarks Store-M used up to 7x less RAM and 5x less disk than a comparable Prometheus deployment holding the same data. Because Store-M is clusterable from day one, no separate long-term storage layer is required.

Pillar 2 - One platform, three signals, one query language

Roshan does not require separate products for metrics, logs and traces. RQL queries can start in metrics, pivot to traces via exemplars and end in log lines, all in one expression. This removes the context-switching cost that teams pay when they run separate tools with separate query syntaxes.

Pillar 3 - Cardinality Shield

High cardinality is the number one cause of observability outages and surprise bills. Cardinality Shield continuously estimates the series count per metric, label and tenant, warns before limits are reached, and can automatically aggregate away abusive labels (for example `user_id` or `request_id`) using pipeline rules.

Pillar 4 - Adaptive downsampling

Older data is automatically rolled up using per-metric strategies (average, max, min, sum, last, percentile sketches). Operators can keep raw data for 15 days, 5-minute resolution for 1 year and 1-hour resolution for 5 years, with transparent query-time resolution selection.

Pillar 5 - Tail-based trace sampling with 100% ingest

The Roshan Gateway buffers complete traces for a configurable decision window (default 30 seconds) and then applies sampling policies: keep all errors, keep all slow traces above the p99 latency, keep 2% of the rest. Operators never lose the interesting traces while cutting storage by up to 90%.

Pillar 6 - Cost-aware log storage

LogLake indexes tokens using bloom filters rather than inverted indexes, which keeps index overhead below 3% of raw volume (versus 30% to 100% for classic full-text indexes). It supports hot, warm and cold tiers on local SSD, network disk and object storage respectively.

Pillar 7 - Automatic correlation

The Correlator builds a live entity graph (services, pods, nodes, deployments, databases, queues). Every telemetry point is attached to one or more entities. When an alert fires, Roshan shows the blast radius, recent deployments, correlated anomalies and the likely root-cause candidates.

Pillar 8 - Open standards first

Roshan natively speaks OpenTelemetry, accepts Prometheus remote write, scrapes Prometheus exporters, understands Loki push, Jaeger and Zipkin formats. Data can be exported via OTLP at any time, so there is no vendor lock-in.

Pillar 9 - Predictable economics

Roshan prices on **gigabytes retained and queried**, not on hosts, containers or custom metrics. Cardinality Shield and tail sampling keep spend predictable, and the Cost Explorer attributes spend to teams and services.

2.2 Summary scorecard (fictional)

Dimension	Roshan 4.2	Typical Prometheus + Grafana stack	Typical SaaS APM
Metrics storage efficiency	Excellent (about 0.4 B/sample)	Good (about 1.3 B/sample)	Opaque
Horizontal scale	Native	Needs Thanos/Mimir/Cortex	Vendor managed
Unified three-signal query	Yes (RQL)	No (PromQL, LogQL, TraceQL)	Partial
Tail sampling	Built in	Separate collector setup	Vendor managed
Cardinality protection	Built in (Shield)	Manual relabeling	Billing surprise
Pricing model	Per GB retained and queried	Infra cost only	Per host / per custom metric
Lock-in risk	Low (OTLP export)	Low	High

2.3 Honest trade-offs

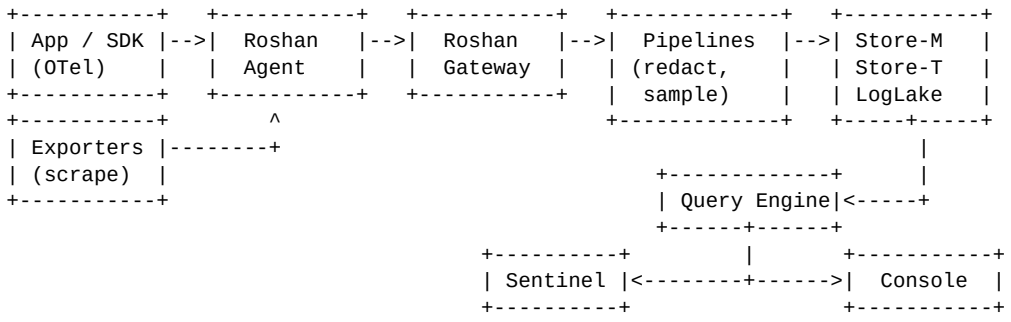
No platform is best at everything. In the interest of balanced documentation, Roshan's known trade-offs are listed here:

- Roshan has a younger ecosystem of community dashboards than Prometheus and Grafana.
- Some exotic PromQL edge cases (for example certain staleness-marker behaviours) differ slightly in RQL. See section 11.4 for a compatibility list.
- The Community edition lacks SSO and multi-tenancy.
- Store-T requires an S3-compatible object store in production.

3. Architecture

3.1 High-level data flow

Telemetry follows the path **Source, Agent, Gateway, Pipelines, Store, Query Engine, Console/API**. Each hop is horizontally scalable and independently deployable.



3.2 The ingest tier

The ingest tier consists of the Roshan Agent and the Roshan Gateway. The agent runs as a DaemonSet (Kubernetes), systemd service (VMs) or sidecar. It performs local buffering on disk (write-ahead queue) so that short network partitions never cause data loss. The gateway is stateless: any gateway replica can accept any request. It performs authentication, tenant resolution, schema validation, per-tenant rate limiting and consistent-hash routing to the correct store shard.

Property	Roshan Agent	Roshan Gateway
Deployment	DaemonSet / sidecar / systemd	Deployment behind L4/L7 load balancer
State	Local disk queue (default 2 GiB)	Stateless; optional tail-sampling buffer in memory
Default ports	4317 (OTLP gRPC), 4318 (OTLP HTTP), 8125 (StatsD)	9443 (ingest), 9444 (admin)
Scaling signal	Per node	CPU and ingest queue depth
Typical resources	100m CPU, 128 MiB RAM	2 vCPU, 4 GiB RAM per 200k samples/s

3.3 The storage tier

Store-M (metrics)

Store-M splits into three roles: **store-m-insert** (accepts writes and shards them), **store-m-storage** (holds data on local disk) and **store-m-select** (serves queries). This split lets operators scale reads and writes independently. Data is replicated with a configurable replication factor (default 2). The storage engine uses a MergeTree-like design: incoming samples land in in-memory parts, are flushed to small parts, and are merged in the background into larger, better compressed parts.

Store-T (traces)

Store-T stores spans in columnar blocks on object storage (S3, GCS, Azure Blob or MinIO). A small write-ahead log (WAL) on local SSD absorbs bursts. Each block has a trace-ID bloom filter and a dictionary-encoded attribute index so that lookups by trace ID are O(1) block touches and attribute searches prune most blocks without reading them.

LogLake (logs)

LogLake partitions logs by tenant and day, then by stream (a unique combination of stream labels such as namespace, app and level). Within each partition, fields are stored columnarly, and a token bloom filter per block accelerates free-text search. Logs in the hot tier live on local SSD, the warm tier on network block storage and the cold tier on object storage.

3.4 The query tier

The Query Engine accepts RQL over HTTP or gRPC. It parses the query, plans it into per-store sub-queries, pushes down filters and aggregations, executes them in parallel, merges the partial results and caches the outcome. A results cache keyed by (tenant, query, step, time-range alignment) reduces repeated dashboard loads by 60% to 90% in the reference workload.

3.5 The control plane

Control-plane services (Sentinel, Correlator, Console backend, Admin API) use a small consensus-backed metadata store (**roshan-meta**, three or five replicas). Control plane outages never block ingestion: gateways cache tenant configuration for up to 24 hours.

3.6 Multi-tenancy model

Every request carries a tenant identifier, resolved from the API key, JWT claim or header `X-Roshan-Tenant`. Tenants are isolated at storage, query and quota level. A tenant may have *workspaces*, which are logical folders for dashboards, alerts and access policies.

3.7 Failure domains and resilience

Failure	Impact	Mitigation
Agent crash	Short collection gap on one node	systemd/Kubernetes restart; disk queue replays
Gateway replica loss	None (stateless)	Load balancer health checks
Store-M storage node loss	None with RF=2	Replica serves reads; background re-sync
Object store outage	Store-T and cold logs unavailable; hot data unaffected	WAL buffers up to 6 hours
Control plane outage	No alert changes, UI degraded; ingest and queries continue	Config cached on gateways
Zone outage	Capacity reduced	Zone-aware replica placement

4. Installation and Getting Started

4.1 Quick start with a single binary

The fastest way to try Roshan is the single-binary Community distribution. It starts all components in one process and stores data under a local directory.

```
# Download and run (fictional commands)
curl -LO https://downloads.roshan.example/4.2.0/roshan-linux-amd64.tar.gz
tar xzf roshan-linux-amd64.tar.gz
./roshan serve --data-dir=./roshan-data --http-addr=:8080

# Console available at http://localhost:8080
# Default OTLP endpoints: grpc://localhost:4317  http://localhost:4318
```

4.2 Running with Docker

```
docker run -d --name roshan \
  -p 8080:8080 -p 4317:4317 -p 4318:4318 \
  -v roshan-data:/var/lib/roshan \
  roshanlabs/roshan:4.2.0 serve --data-dir=/var/lib/roshan
```

4.3 Kubernetes with Helm

```
helm repo add roshan https://charts.roshan.example
helm repo update
helm install roshan roshan/roshan-stack \
  --namespace observability --create-namespace \
  --set global.tenant=default \
  --set storeM.storage.size=500Gi \
  --set storeT.objectStore.bucket=my-roshan-traces \
  --set loglake.hot.size=300Gi
```

4.4 System requirements

Deployment size	Active series	Ingest rate	Suggested nodes
Small	up to 500k	up to 50k samples/s	3 nodes x 4 vCPU x 16 GiB
Medium	up to 10M	up to 1M samples/s	9 nodes x 8 vCPU x 32 GiB
Large	up to 100M	up to 8M samples/s	30 nodes x 16 vCPU x 64 GiB
Extra large	above 100M	above 8M samples/s	Contact sizing guide, section 14

4.5 Installing the agent

```
# Linux (systemd)
curl -sSL https://get.roshan.example/agent.sh | \
  ROSHAN_GATEWAY=https://gw.example.com:9443 ROSHAN_API_KEY=rk_live_xxx sh

# Kubernetes DaemonSet
kubectl apply -f https://get.roshan.example/agent/k8s.yaml
```

4.6 Sending your first telemetry

Metrics via Prometheus remote write

```
# prometheus.yml
remote_write:
  - url: https://gw.example.com:9443/api/v1/write
    headers:
      X-Roshan-Tenant: default
    authorization:
      credentials: rk_live_xxx
```

Traces via OpenTelemetry SDK (environment variables)

```
export OTEL_EXPORTER_OTLP_ENDPOINT=https://gw.example.com:9443
export OTEL_EXPORTER_OTLP_HEADERS="authorization=Bearer rk_live_xxx"
export OTEL_SERVICE_NAME=checkout-service
export OTEL_RESOURCE_ATTRIBUTES=deployment.environment=prod,team=payments
```

Logs via the Roshan Agent

```
# /etc/roshan/agent.yaml
logs:
  sources:
    - type: file
      include: ["/var/log/app/*.log"]
      labels: { app: "billing", env: "prod" }
```

4.7 Verifying the installation

- Open the Console and visit **Settings, Health**; every component should show green.
- Run `roshanctl status` and confirm all components report *ready*.
- In Explore, run `roshan_ingest_samples_total` to confirm self-monitoring data.
- Send a test trace with `roshanctl trace test` and search for it.

5. Metrics

5.1 Metrics data model

A Roshan metric is a named series of timestamped floating-point values identified by a set of labels. The model is fully compatible with the Prometheus data model, with two additions: **exemplars** (links to trace IDs) are first-class and indexed, and **native histograms** are stored with sparse bucket compression.

Type	Description	Example
Counter	Monotonically increasing value; resets handled at query time.	http_requests_total
Gauge	Value that can go up and down.	memory_working_set_bytes
Histogram	Bucketed distribution; classic and native supported.	http_request_duration_seconds
Summary	Client-side quantiles (discouraged; no aggregation).	rpc_latency_summary
Sketch	Roshan DDSketch-based percentile type for accurate cross-instance percentiles.	checkout_latency_sketch

5.2 Why Victoria Metrics instead of a classic TSDB

Roshan chose VictoriaMetrics technology for Store-M after evaluating several storage engines. The following table summarizes the technical reasons and the measured effect in Roshan's fictional reference benchmark (10 million active series, 15-second scrape interval, 30-day retention).

Criterion	Prometheus TSDB	Roshan Store-M (VictoriaMetrics-based)
Storage layout	Two-hour blocks plus WAL, head block in memory	MergeTree-like parts with background merges
Bytes per sample on disk	About 1.3 bytes	About 0.4 bytes
RAM at 10M active series	About 60 GiB	About 9 GiB
Horizontal scaling	Not native (needs Thanos, Mimir, Cortex)	Native cluster mode
Long-term retention	Needs object-store sidecar	Built in with tiering
High churn handling	Head block grows; compaction stalls	Tolerates churn; indexes are per-day
Downsampling	Not built in	Built in, adaptive
Query speed (30-day range, p95)	9.4 s	1.8 s
Write path	Single node, WAL fsync	Sharded, replicated, batched
Back-fill of old samples	Limited by head window	Supported within retention

> **Note:** The figures above are fictional benchmark values included only to give the documentation agent concrete numbers to retrieve and compare.

5.3 Ingestion paths

Protocol	Endpoint / port	Notes
Prometheus remote write v1/v2	POST /api/v1/write on 9443	Snappy protobuf; exemplars supported in v2.
OTLP metrics (gRPC)	4317	Delta and cumulative temporality both accepted.
OTLP metrics (HTTP)	4318 /v1/metrics	Protobuf or JSON.
Prometheus scrape	Agent scrape config	Service discovery: Kubernetes, Consul, EC2, DNS, file.
StatsD / DogStatsD	UDP 8125	Tags converted to labels.
InfluxDB line protocol	POST /influx/write	Fields become metric suffixes.

Protocol	Endpoint / port	Notes
Graphite plaintext	TCP 2003	Dot paths converted via mapping rules.

5.4 Scraping configuration

```
# /etc/roshan/agent.yaml (metrics section)
metrics:
  scrape_interval: 15s
  scrape_timeout: 10s
  jobs:
    - name: kubernetes-pods
      discovery: kubernetes_sd
      relabel:
        - action: keep
          source: __meta_kubernetes_pod_annotation_roshan_scrape
          regex: "true"
    - name: node-exporters
      static_targets: ["10.0.0.11:9100", "10.0.0.12:9100"]
```

5.5 Cardinality management (Cardinality Shield 2.0)

Cardinality is the number of unique label combinations for a metric. Roshan tracks it per tenant, per metric and per label using HyperLogLog sketches. Cardinality Shield 2.0, introduced in version 4.2, adds predictive warnings and automatic remediation.

Shield stages

- **Observe:** collect statistics with no enforcement.
- **Warn:** emit alerts and Console banners at 70% and 90% of the limit.
- **Aggregate:** automatically drop the offending label and re-aggregate (for example drop `pod_uid` and sum).
- **Reject:** refuse new series above the hard limit; existing series continue.

```
# Pipeline rule: aggregate away request_id automatically
pipelines:
  metrics:
    - name: strip-request-id
      match: { label_present: request_id }
      action: drop_label
      labels: [request_id]
      aggregate: sum
```

Limit	Community	Team	Enterprise
Active series per tenant	1,000,000	20,000,000	Configurable
New series per hour per tenant	500,000	5,000,000	Configurable
Label name length	128	128	256
Label value length	1,024	2,048	4,096
Labels per series	30	40	60

5.6 Retention and adaptive downsampling

Retention is defined by tier policies. Each policy specifies a resolution and a duration. Queries transparently choose the finest resolution that covers the requested range unless the user forces a resolution.

```
retention:
  policies:
    - name: raw
      resolution: native
      keep: 15d
    - name: five-minute
      resolution: 5m
      keep: 400d
      aggregations: [avg, min, max, sum, count, last]
    - name: hourly
      resolution: 1h
      keep: 5y
      aggregations: [avg, max, p99_sketch]
```

Resolution	Typical use	Storage per series per year
15 s (raw)	Incident debugging, SLO burn alerts	About 5.2 MB
5 m	Capacity planning, weekly review	About 0.3 MB
1 h	Quarterly and annual trends	About 0.03 MB

5.7 RQL for metrics

RQL is a superset of PromQL and MetricsQL. Every valid PromQL query runs unmodified. RQL adds convenience functions and cross-signal operators.

Basic queries

```
# Request rate per service
sum by (service) (rate(http_requests_total[5m]))

# Error ratio
sum(rate(http_requests_total{status=~"5.."}[5m]))
  / sum(rate(http_requests_total[5m]))

# p99 latency from a native histogram
histogram_quantile(0.99, sum by (le, service) (rate(http_request_duration_seconds[5m])))
```

RQL extensions

Function / operator	Purpose	Example
rollup_rate()	Rate with min/max/avg over the window	rollup_rate(requests_total[5m])
anomaly()	Z-score style anomaly band	anomaly(cpu_usage, sensitivity=3)
forecast()	Linear/seasonal forecast	forecast(disk_used_bytes[7d], 24h)
topk_by()	Top K with tie handling	topk_by(5, service, rate(errors_total[5m]))
exemplars()	Return exemplar trace IDs for a series	exemplars(http_latency_bucket{le='1'})
to_traces()	Pivot from metric series to traces	to_traces(exemplars(...))
to_logs()	Pivot from a metric series to logs by labels	to_logs(errors_total{service='api'})
share()	Fraction of total per group	share(sum by (team)(cost_usd))

5.8 Recording rules and derived metrics

```
rules:
  - name: slo-checkout
    interval: 30s
    rules:
      - record: service:request_error_ratio:rate5m
        expr: |
          sum by (service) (rate(http_requests_total{status=~"5.."}[5m]))
          / sum by (service) (rate(http_requests_total[5m]))
```

Rules are evaluated by Sentinel, written back to Store-M and tagged with `__source__="rule"`. Rule evaluation lag is exposed as `roshan_rule_eval_lag_seconds`.

5.9 Exemplars

Exemplars link a metric sample to a trace. When an instrumented application records a histogram observation inside an active span, the SDK attaches the trace and span IDs. Store-M keeps an exemplar index so the Console can render a dot on a latency chart that opens the underlying trace in one click.

5.10 Self-monitoring metrics

Metric	Type	Meaning
roshan_ingest_samples_total	Counter	Samples accepted per tenant
roshan_ingest_rejected_total	Counter	Samples rejected, labelled by reason
roshan_active_series	Gauge	Active series per tenant
roshan_storem_merge_duration_seconds	Histogram	Background merge duration
roshan_query_duration_seconds	Histogram	RQL query latency
roshan_query_cache_hit_ratio	Gauge	Result cache effectiveness
roshan_agent_queue_bytes	Gauge	Agent disk queue size
roshan_gateway_inflight_requests	Gauge	Concurrent ingest requests
roshan_shield_dropped_series_total	Counter	Series dropped by Cardinality Shield

5.11 Metrics comparison: Roshan vs Prometheus

Capability	Prometheus	Roshan
Storage engine	Local TSDB	VictoriaMetrics-based Store-M
Clustering	Federation or external projects	Native
Long-term storage	External	Built in
Downsampling	None	Adaptive, per metric
Multi-tenancy	None	Native, quota-enforced
Query language	PromQL	RQL (PromQL superset)
Exemplars	Experimental, limited	First-class, indexed
Native histograms	Supported	Supported, sparse compression
Cardinality control	Relabeling and limits	Shield 2.0 with auto aggregation
Backfilling	Limited	Supported
Resource use at 10M series	High	Low
Maturity and ecosystem	Very large	Growing

6. Traces

6.1 Concepts

A **trace** represents one request as it flows through a distributed system. It consists of **spans**, each with a name, start time, duration, status, attributes, events and links. Roshan accepts spans in OpenTelemetry format and stores them in Store-T.

Term	Definition
Trace ID	128-bit identifier shared by all spans in a request.
Span ID	64-bit identifier of one operation.
Parent span	The span that caused this span; the root has none.
Resource attributes	Describe the emitter: service.name, k8s.pod.name, host.id.
Span attributes	Describe the operation: http.method, db.statement, user.tier.
Span kind	SERVER, CLIENT, PRODUCER, CONSUMER or INTERNAL.
Span events	Timestamped annotations such as exceptions.
Span links	References to causally related spans in other traces.

6.2 Instrumentation options

- **OpenTelemetry SDKs** for Java, Go, Python, Node.js, .NET, Ruby, PHP, Rust and C++.
- **Roshan eBPF auto-instrumentation** in the agent: zero code changes for HTTP, gRPC, MySQL, PostgreSQL, Redis and Kafka traffic on Linux kernels 5.8 and above.
- **Service mesh integration** with Istio and Linkerd sidecars exporting spans.
- **Jaeger and Zipkin** compatible receivers for legacy clients.

6.3 Ingestion and tail-based sampling

The gateway groups spans by trace ID using consistent hashing, so all spans of a trace reach the same gateway replica. The replica waits up to the decision window (default 30 seconds) and evaluates the sampling policies in order.

```
sampling:
  decision_window: 30s
  max_buffered_traces: 500000
  policies:
    - name: keep-errors
      type: status_code
      status: ERROR
      action: keep
    - name: keep-slow
      type: latency
      threshold_ms: 2000
      action: keep
    - name: keep-vip-tenants
      type: attribute
      key: customer.tier
      values: [enterprise]
      action: keep
    - name: baseline
      type: probabilistic
      percent: 2
      action: keep
```

Sampling strategy	Where it happens	Pros	Cons
Head-based probabilistic	SDK	Cheap, simple	May drop interesting traces
Tail-based (Roshan default)	Gateway	Keeps errors and slow traces	Needs buffering memory
Adaptive rate	Gateway	Stays within a budget per service	Slightly less deterministic
No sampling	N/A	Complete data	Highest cost

6.4 Storage format and indexing

Store-T writes blocks of about 128 MiB. Each block contains span data in columnar form (trace ID, span ID, parent ID, name, kind, start, duration, status and dictionary-encoded attributes). Every block has a footer with bloom filters on trace ID and indexed attributes. Compaction merges small blocks nightly and re-sorts by service and time to improve pruning.

Metric	Roshan Store-T (reference)
Compression ratio	About 9x versus raw OTLP
Trace lookup by ID (p95)	Under 150 ms for 7 days of data
Attribute search (p95)	Under 2 s over 24 hours
Ingest per gateway replica	About 120,000 spans/s
Default retention	14 days (configurable per tenant)

6.5 Searching and querying traces

```
# Find slow checkout traces with errors
traces { service="checkout" && status=error && duration > 1s }

# Aggregate span durations by endpoint
traces { service="api" } | rate() by (http.route)

# p95 span duration per downstream dependency
traces { span.kind="client" } | quantile(duration, 0.95) by (peer.service)

# Pivot from a trace to its logs
traces { trace_id="4bf92f3577b34da6a3ce929d0e0e4736" } | to_logs()
```

6.6 Service map and dependency graph

The Correlator derives a service map from span relationships. Edges show request rate, error rate and p95 latency, and nodes can be colored by SLO burn. Maps are available per time range and support diffing two time windows, which is useful for comparing before and after a deployment.

6.7 Span metrics (RED metrics from traces)

Roshan can generate Rate, Errors and Duration metrics directly from spans, without separate instrumentation. These appear in Store-M as `span_calls_total` and `span_duration_seconds` with labels `service`, `operation`, `kind` and `status`. Because the generation happens before sampling, the metrics stay accurate even when only a small fraction of traces is retained.

6.8 Continuous profiling (preview)

Version 4.0 added a preview of continuous profiling. Profiles from the agent (CPU and heap, collected through eBPF or pprof endpoints) are linked to spans so that a slow span can open a flame graph for the same period and process.

6.9 Traces comparison: Roshan vs Jaeger vs Tempo

Capability	Jaeger	Grafana Tempo	Roshan Store-T
Storage backend	Cassandra, Elasticsearch, ClickHouse, etc.	Object storage	Object storage + WAL
Search by attributes	Index-based, costly at scale	TraceQL, block scanning	RQL traces with attribute index
Tail sampling	Via collector	Via collector	Built into gateway
Span metrics	Via separate processor	Metrics generator	Built in, pre-sampling
Service map	Basic	Via Grafana and metrics	Native, diffable
Exemplar linking	None	Via Grafana	Native
Log correlation	Manual	Via Grafana and Loki	Native
Multi-tenancy	Limited	Yes	Yes, with quotas
Profiling link	None	Via Pyroscope	Preview
Operational complexity	Medium to high	Medium	Low to medium

7. Logs

7.1 Log data model

A log record in Roshan has a timestamp, a body, severity, resource attributes, optional trace and span IDs and a set of structured fields. Stream labels (such as `namespace`, `app`, `env`) define the stream. Everything else is stored as fields, which can be filtered without being indexed up front (schema-on-read).

7.2 Ingestion paths

Source	Mechanism	Notes
Files and containers	Agent file tailer / Kubernetes log discovery	Multiline and rotation aware.
OTLP logs	4317 / 4318	Trace IDs preserved.
Syslog	TCP/UDP 514, TLS 6514	RFC 3164 and RFC 5424.
Loki push API	POST /loki/api/v1/push	Drop-in for Promtail and Alloy clients.
Elasticsearch bulk API	POST /insert/elasticsearch/_bulk	For Filebeat and Logstash.
Journald	Agent source	Systemd units as labels.
Cloud services	Kinesis, Pub/Sub, Event Hubs connectors	Pull-based.

7.3 Parsing and enrichment pipelines

```
pipelines:
  logs:
    - name: nginx-access
      match: { app: nginx }
      steps:
        - parse_regex:
            field: body
            pattern: '^(?P<client>\S+) \S+ \S+ \[(?P<ts>[^\]]+)\] "(?P<method>\S+) (?P<path>\S+)'
        - parse_json: { field: body, optional: true }
        - set_timestamp: { field: ts, format: "02/Jan/2006:15:04:05 -0700" }
        - add_labels: { team: "edge" }
        - redact:
            patterns: [credit_card, email, aws_secret_key]
            replacement: "[REDACTED]"
```

7.4 Indexing strategy

LogLake avoids full inverted indexes. Instead each block carries a token bloom filter and min/max statistics for numeric fields. A query such as `"timeout" and status>=500` first prunes by time and stream, then by bloom filter, and finally scans the surviving blocks with vectorized filters. This design yields index overhead below 3% of raw size.

Approach	Index overhead	Ingest cost	Ad-hoc search speed
Full inverted index (typical Elasticsearch)	30% to 100%	High	Very fast
Labels-only index (typical Loki)	Under 1%	Low	Slow for free text
Token bloom + columnar (Roshan LogLake)	Under 3%	Low	Fast

7.5 Querying logs (RQL-Logs)

```
# All errors for an app in the last hour
logs { app="payments" } |= "error" | level="error"

# Parse JSON and filter on a field
logs { app="api" } | json | latency_ms > 500

# Count errors per service over time
logs { env="prod" } | level="error" | rate(5m) by (service)

# Top 10 error messages
logs { app="checkout" } | level="error" | top 10 by (message)

# Pivot to the trace for a log line
logs { app="checkout" } | trace_id != "" | to_traces()
```

7.6 Log-to-metric conversion

Frequently used log patterns can be converted to metrics at ingest time. The metrics are cheap to store and fast to query, and the raw logs can then be sampled or short-lived.

```
pipelines:
  logs:
    - name: login-failures
      match: { app: auth }
      steps:
        - metric:
            name: auth_login_failures_total
            type: counter
            when: 'body contains "login failed"'
            labels: [service, region]
```

7.7 Retention tiers

Tier	Medium	Default age	Query latency (typical)
Hot	Local NVMe SSD	0 to 7 days	Under 1 s
Warm	Network block storage	7 to 30 days	1 to 5 s
Cold	Object storage	30 days to 1 year or more	5 to 30 s
Archive	Glacier-class storage	Over 1 year	Restore required (hours)

7.8 Sensitive data handling

- Redaction patterns run in Pipelines before storage, so sensitive data never lands on disk.
- Field-level access control (Enterprise) hides chosen fields from specific roles.
- Right-to-erasure workflows delete records matching a filter across all tiers.
- Every query against designated sensitive streams is recorded in the audit log.

7.9 Logs comparison: Roshan vs Loki vs Elasticsearch vs Splunk

Capability	Loki	Elasticsearch	Splunk	Roshan LogLake
Indexing model	Labels only	Full inverted	Full inverted	Token bloom + columnar
Storage cost	Low	High	Very high	Low
Free-text search	Slower	Fast	Fast	Fast
Query language	LogQL	KQL / Lucene / ES|QL	SPL	RQL-Logs
Compression	5x to 10x	2x to 4x	3x to 5x	12x to 20x
Object storage tiers	Yes	Frozen tier	SmartStore	Yes, 3 tiers + archive

Capability	Loki	Elasticsearch	Splunk	Roshan LogLake
Log-to-metric	Ruler recording rules	Transforms	Summary indexing	Pipeline step
Trace correlation	Via Grafana	Via APM	Via add-ons	Native
Operational burden	Low to medium	High	Medium to high	Low

8. Correlation: One Click Across Signals

8.1 The entity graph

The Correlator maintains a live graph of entities: services, deployments, pods, nodes, databases, queues, load balancers and cloud resources. Edges represent relationships such as *calls*, *runs on*, *owns* and *depends on*. Every metric series, span and log record is attached to the entities named by its resource attributes.

8.2 Correlation keys

Key	Links	Mechanism
trace_id	Trace to logs	Log records carry the active trace ID.
exemplar	Metric to trace	Histogram sample references trace and span IDs.
resource attributes	All signals to entities	service.name, k8s.pod.uid, host.id.
deployment marker	Changes to symptoms	CI/CD webhook creates a marker on charts.
span link	Async traces	Producer to consumer relationships.

8.3 Investigation workflow

- An alert fires: *checkout p99 latency above 1.5 s*.
- Sentinel opens an incident and the Correlator lists affected entities and recent deployments.
- The Console shows exemplar dots on the latency chart; clicking one opens the slowest trace.
- The trace waterfall highlights a 1.2 s database span; a link opens that pod's logs for the same window.
- The log view shows connection pool exhaustion; the deployment marker reveals a config change 8 minutes earlier.

8.4 Cross-signal RQL example

```
# From high-latency metrics to the logs of the slowest traces
logs { app="checkout" }
  | trace_id in (
      to_traces(exemplars(http_request_duration_seconds_bucket{service="checkout"} > 1.5))
    )
  | level >= "warn"
```

8.5 Root-cause candidates

The Correlator ranks candidate causes using three signals: temporal proximity to the symptom, topology distance in the entity graph and statistical anomaly strength. It never claims a definitive root cause; instead it presents the top five candidates with evidence links so engineers can verify them.

9. Alerting with Roshan Sentinel

9.1 Alert types

Type	Evaluated on	Example
Threshold	Metrics	CPU above 90% for 10 minutes
Rate / ratio	Metrics	Error ratio above 2% for 5 minutes
SLO burn rate	Metrics	Fast burn: 14.4x over 1 hour and 5 minutes
Anomaly (new in 4.2)	Metrics	Traffic deviates 4 sigma from seasonal baseline
Log pattern	Logs	More than 20 'OutOfMemory' lines in 5 minutes
Trace based	Traces	p95 of span duration for payments-db above 800 ms
Absence	Any	No heartbeat from billing-worker for 3 minutes
Composite	Multiple signals	High latency AND rising log errors

9.2 Defining an alert rule

```
alerts:
  - name: CheckoutHighErrorRatio
    expr: |
      sum(rate(http_requests_total{service="checkout",status=~"5.."}[5m]))
      / sum(rate(http_requests_total{service="checkout"}[5m])) > 0.02
    for: 5m
    severity: critical
    labels: { team: payments }
    annotations:
      summary: "Checkout error ratio is {{ $value | humanizePercentage }}"
      runbook: "https://runbooks.example.com/checkout-errors"
```

9.3 Routing, grouping and silencing

Alerts are grouped by labels (default `alertname`, `service`, `env`) and routed through a tree of matchers to receivers. Sentinel supports Slack, Microsoft Teams, PagerDuty, Opsgenie, email, SMS, webhooks and ServiceNow.

```
routing:
  receiver: default-slack
  group_by: [alertname, service, env]
  group_wait: 30s
  group_interval: 5m
  repeat_interval: 4h
  routes:
    - match: { severity: critical }
      receiver: pagerduty-oncall
    - match: { team: payments }
      receiver: payments-slack
```

9.4 Inhibition and dependency awareness

Sentinel uses the entity graph to suppress downstream noise automatically. If a database is down, alerts from services that depend on it are grouped under the database incident instead of paging each owner separately. Operators can disable this behaviour per route.

9.5 Anomaly-aware alerts

Anomaly alerts learn a seasonal baseline (hourly, daily, weekly components) for each series from the previous four weeks. They require at least 14 days of data and are evaluated using the `anomaly()` RQL function. The sensitivity parameter ranges from 1 (very sensitive) to 6 (very conservative).

9.6 Incident management

- Incidents are created automatically from critical alert groups.
- Each incident holds a timeline of alerts, deployments, annotations and chat messages.
- Post-incident reports can be exported to Markdown and PDF.
- Mean time to acknowledge (MTTA) and mean time to resolve (MTTR) are tracked per team.

9.7 Alerting comparison

Capability	Prometheus + Alertmanager	Grafana Alerting	Datadog Monitors	Roshan Sentinel
Metric alerts	Yes	Yes	Yes	Yes
Log alerts	No	Via Loki ruler	Yes	Yes, native
Trace alerts	No	Limited	Yes	Yes, native
Anomaly detection	No	Plugin	Yes	Yes (4.2)
Dependency-aware suppression	Manual inhibit rules	Manual	Partial	Automatic via entity graph
Incident timeline	No	Add-on	Yes	Yes
Config as code	YAML	YAML / provisioning	Terraform	YAML, Terraform, GitOps

10. Dashboards and the Roshan Console

10.1 Console areas

Area	Purpose
Explore	Ad-hoc RQL queries across metrics, traces and logs with split views.
Dashboards	Grids of panels with variables, annotations and drill-downs.
Service Map	Live dependency graph with RED metrics and SLO overlay.
Alerts and Incidents	Rule list, active alerts, silences, incident timelines.
Cost Explorer	Ingest and storage spend by team, service, signal and label.
Admin	Tenants, users, API keys, quotas, retention and audit log.

10.2 Panel types

- Time series, bar, stat, gauge, heatmap and histogram panels.
- Table and top-N panels with column formatting.
- Trace waterfall and flame graph panels.
- Log stream and log volume panels with field browser.
- Topology panels showing the entity graph.
- Text/Markdown and iframe panels.

10.3 Dashboards as code

```
# dashboards/checkout.yaml
apiVersion: roshan.example/v1
kind: Dashboard
metadata: { name: checkout-overview, workspace: payments }
spec:
  variables:
    - name: env
      query: label_values(http_requests_total, env)
  panels:
    - title: Request rate
      type: timeseries
      query: sum(rate(http_requests_total{service="checkout",env="$env"}[5m]))
    - title: p99 latency
      type: timeseries
      query: histogram_quantile(0.99, sum by (le)(rate(http_request_duration_seconds{service="checkout"}[5m])))
```

10.4 Grafana compatibility

Roshan exposes a Prometheus-compatible query API, a Loki-compatible API and a Tempo-compatible API. Existing Grafana installations can add Roshan as a data source of each type without changes to existing dashboards. The Console can also import Grafana dashboard JSON, converting most panels automatically.

10.5 Sharing and embedding

- Public snapshot links with expiry.
- Signed embed URLs for internal portals.
- Scheduled PDF reports delivered by email.
- Annotations from CI/CD systems through the Events API.

11. RQL: Roshan Query Language

11.1 Design goals

- Run every valid PromQL expression unchanged.
- Provide MetricsQL conveniences (for example default lookbehind windows, keep_metric_names).
- Give logs and traces first-class syntax with a shared label-matcher language.
- Allow safe pivots between signals using functions rather than separate tools.

11.2 Syntax summary

Signal	Selector form	Pipeline form	Example
Metrics	metric{label="x"}	functions and operators	rate(errors_total{app="api"}[5m])
Logs	logs { label="x" }	| filter | parse | aggregate	logs { app="api" } |= "timeout"
Traces	traces { attr="x" }	| aggregate | pivot	traces { service="api" } | rate()

11.3 Operators and functions

Category	Functions
Aggregation	sum, avg, min, max, count, stddev, quantile, topk, bottomk, topk_by, share
Range vector	rate, irate, increase, delta, deriv, rollup_rate, avg_over_time, max_over_time, quantile_over_time
Histogram	histogram_quantile, histogram_avg, histogram_share, histogram_fraction
Time	time, timestamp, day_of_week, hour, offset, at
Prediction	predict_linear, forecast, anomaly
Cross-signal	exemplars, to_traces, to_logs, to_metrics
Logs	json, logfmt, regexp, pattern, line_format, unwrap, top
Traces	duration, status, parent, child, descendant, quantile, rate

11.4 PromQL compatibility notes

Behaviour	PromQL	RQL
Staleness markers	Series becomes stale after missing scrape	Same; default lookbehind configurable per tenant
Implicit window	Range required in rate()	Window optional; defaults to max(step, 4 x scrape interval)
Metric name retention after functions	Dropped	Dropped by default; use keep_metric_names
Native histograms	Supported	Supported
@ modifier	Supported	Supported
Subquery step default	Global evaluation interval	Query step
NaN handling in comparisons	NaN is false	Same
Offset on aggregation	Not allowed	Allowed as extension

11.5 Query limits and protection

Limit	Default	Purpose
max_query_duration	60 s	Cancel long running queries
max_series_per_query	500,000	Protect memory
max_samples_per_query	2,000,000,000	Protect CPU
max_log_bytes_scanned	200 GiB	Prevent runaway log scans
max_concurrent_queries_per_tenant	16	Fair sharing
max_lookback	Retention horizon	Bound time range

11.6 Caching and query acceleration

Roshan uses a multi-level cache: a results cache for repeated queries, an index cache for label lookups, and a chunk cache for hot blocks. A query splitter breaks long range queries into day-aligned sub-queries that are cached independently, so a dashboard open for a week reuses six days of results on each refresh.

12. Comparisons with Other Platforms

This chapter compares Roshan with commonly used observability tools. Information about Roshan is fictional. Statements about other products are generalized descriptions for the purposes of this synthetic document and are not authoritative; a documentation agent should treat the Roshan side of every table as the source of truth.

12.1 Roshan vs Prometheus

Area	Prometheus	Roshan	Winner (per Roshan docs)
Primary purpose	Metrics monitoring and alerting	Unified metrics, traces, logs	Roshan
Storage engine	Local TSDB	VictoriaMetrics-based Store-M	Roshan
Scalability	Vertical, federation	Horizontal, native	Roshan
Retention	Typically 15 days to months	Years with downsampling	Roshan
Setup simplicity (small scale)	Very simple single binary	Simple single binary	Tie
Ecosystem size	Very large	Growing	Prometheus
Pull model	Core design	Supported via agent	Tie
Cost at scale	Needs extra systems	Lower infra and ops cost	Roshan
Query language	PromQL	RQL superset	Roshan
Community and CNCF status	Graduated project	Vendor-led open core	Prometheus

12.2 Roshan vs Thanos, Cortex and Mimir

Area	Thanos	Cortex / Mimir	Roshan
Architecture	Sidecar plus store gateway plus compactor	Microservices with ring and object store	Store-M insert, storage, select
Number of components for HA metrics	6 or more	8 or more	3 plus gateway
Long-term storage	Object store	Object store	Local tiers with optional object store
Downsampling	Yes, fixed 5m and 1h	Limited	Adaptive, configurable
Ingest efficiency	Depends on Prometheus	Good	Excellent
Operational burden	High	High	Medium
Native logs and traces	No	No	Yes

12.3 Roshan vs the Grafana stack (Mimir, Loki, Tempo)

Area	Grafana LGTM stack	Roshan
Products to operate	Grafana, Mimir, Loki, Tempo (plus Alloy, Pyroscope)	One platform
Query languages	PromQL, LogQL, TraceQL	RQL (all signals)
Cross-signal pivots	Via Grafana UI configuration	Native in query and UI
Metrics store	Mimir	VictoriaMetrics-based Store-M
Log index	Labels only	Token bloom + columnar
Trace sampling	Collector config	Gateway built in
Dashboards	Excellent, huge plugin library	Good, Grafana-compatible APIs
Licensing	AGPL (core) plus enterprise	Open core, source-available components

Area	Grafana LGTM stack	Roshan
Best fit	Teams that want modular best of breed	Teams that want an integrated platform

12.4 Roshan vs Datadog

Area	Datadog	Roshan
Delivery	SaaS only	SaaS, self-hosted, edge
Pricing basis	Per host, per custom metric, per GB logs, per span	Per GB retained and queried
Data ownership	Vendor cloud	Customer choice
Integrations	Over 700	About 240 plus all OpenTelemetry and Prometheus exporters
APM	Very mature	Mature; profiling in preview
Real user monitoring	Yes	Planned for 5.0
Lock-in	High (proprietary agent and formats)	Low (OTLP, PromQL-compatible)
Custom metric cost risk	High	Controlled by Cardinality Shield

12.5 Roshan vs Elastic Observability and Splunk

Area	Elastic Observability	Splunk Observability / Enterprise	Roshan
Core engine	Lucene inverted index	Proprietary indexers / SignalFx	VictoriaMetrics, columnar logs
Strength	Log search and analytics	Enterprise security and logs	Unified cost-efficient telemetry
Resource use for logs	High	High	Low
Metrics efficiency	Moderate	Good	Excellent
Licensing complexity	Tiered subscriptions	Ingest or workload based	GB retained and queried
Security analytics (SIEM)	Strong	Very strong	Basic detections

12.6 Roshan vs New Relic, Dynatrace, SigNoz and OpenObserve

Area	New Relic	Dynatrace	SigNoz	OpenObserve	Roshan
Model	SaaS	SaaS / managed	Open source + cloud	Open source + cloud	Open core + cloud
Store	NRDB	Grail	ClickHouse	Parquet on object store	Victoria-based + columnar
Pricing basis	Per user and GB	Per host unit	Per GB	Per GB	Per GB retained/queried
Auto-instrumentation	Yes	OneAgent, strong	OTel	OTel	eBPF + OTel
AI-assisted RCA	Yes	Davis AI	Limited	Limited	Correlator ranking
Self-host option	No	Limited	Yes	Yes	Yes

12.7 Benchmark summary (fictional)

The Roshan Labs reference benchmark used 10 million active series, 200,000 spans per second, 500 GB of logs per day, a 30-day retention window and identical hardware (AWS-class 16 vCPU nodes). Results below are synthetic.

Metric	Prometheus + Thanos	Grafana LGTM	Roshan 4.2
Metrics ingest throughput	1.1 M samples/s	4.8 M samples/s	18 M samples/s
Metrics storage for 30 days	4.1 TB	2.3 TB	0.9 TB
p95 dashboard query (7 days)	5.6 s	2.9 s	1.1 s
Log storage for 30 days (500 GB/day)	N/A	2.1 TB	1.0 TB
Trace lookup by ID p95	N/A	420 ms	150 ms
Nodes required	36	28	14
Monthly infra cost index	100	74	38

12.8 Decision guide

If you need...	Choose
A tiny single-node metrics setup with maximum community docs	Prometheus
Best-of-breed modular stack with strong dashboards	Grafana LGTM
Unified, cost-efficient metrics, traces and logs with one query language	Roshan
Full SIEM and compliance search	Splunk or Elastic
Fully managed SaaS with huge integration catalog	Datadog or Dynatrace
Edge and IoT observability in a single binary	Roshan Edge

13. Security, Access Control and Multi-Tenancy

13.1 Authentication

Method	Used by	Notes
API keys	Agents, gateways, CI	Scoped to tenant and signal; rotate every 90 days.
OIDC SSO	Console users	Okta, Azure AD, Google, Keycloak.
SAML 2.0	Console users (Team and up)	Group claims mapped to roles.
mTLS	Agent to gateway	Optional, recommended for production.
Service accounts	Automation	Short-lived tokens via OIDC federation.

13.2 Role-based access control

Role	Permissions
Viewer	Read dashboards, run queries within allowed workspaces.
Editor	Viewer plus create and edit dashboards, alerts and pipelines.
Operator	Editor plus manage silences, incidents and quotas.
Admin	Operator plus manage users, API keys, retention and SSO.
Auditor	Read-only access to audit logs and configuration.
Billing	Cost Explorer and invoices only.

13.3 Data protection

- Encryption in transit with TLS 1.2 or 1.3 on every network hop.
- Encryption at rest using AES-256 with customer-managed keys (Enterprise).
- Field-level redaction in Pipelines and field-level access control in LogLake.
- Data residency: tenants can be pinned to a region; cross-region replication is opt-in.
- Audit logs record logins, queries on sensitive streams, configuration changes and exports.

13.4 Compliance (fictional)

Roshan Cloud claims SOC 2 Type II, ISO 27001, GDPR and HIPAA-eligible operation. Self-hosted deployments inherit the compliance posture of the operator's environment.

13.5 Quotas and fair use

Quota	Scope	Default (Team)
Ingest rate (samples/s)	Tenant	500,000
Ingest rate (log bytes/s)	Tenant	20 MiB/s
Ingest rate (spans/s)	Tenant	100,000
Query concurrency	Tenant	16
Dashboards per workspace	Workspace	500
Alert rules per tenant	Tenant	5,000

14. Deployment, Sizing and Operations

14.1 Reference topologies

Topology	Description	When to use
Single binary	All components in one process	Dev, test, edge
Small HA	3 nodes, each running every role	Up to 500k series
Split roles	Dedicated insert, storage, select nodes	Medium to large
Multi-cluster	Regional clusters with global query layer	Global orgs
Hybrid	Edge agents buffering to a central cloud	IoT and retail

14.2 Sizing rules of thumb

Signal	Rule of thumb
Metrics	1 vCPU per 150k samples/s ingested; 1 GiB RAM per 1M active series; 0.4 bytes per sample on disk.
Traces	1 vCPU per 25k spans/s; plan 150 bytes per sampled span in object storage.
Logs	1 vCPU per 15 MB/s; expect 12x to 20x compression; 3% index overhead.
Query tier	1 vCPU per 4 concurrent queries; double for heavy log scans.
Gateway	1 vCPU per 100k samples/s or 25k spans/s or 10 MB/s logs.

14.3 Capacity planning example

An organization runs 800 Kubernetes nodes emitting 6 million active series scraped every 15 seconds, 40,000 spans per second (after 10% head sampling) and 120 GB of logs per day. The sizing guide produces the following plan.

Resource	Calculation	Result
Metrics ingest	6,000,000 / 15 s	400,000 samples/s
Store-M CPU	400k / 150k	About 3 vCPU, round to 8 with headroom
Store-M RAM	6M series / 1M x 1 GiB x 2 replicas	About 12 GiB plus 50% headroom
Store-M disk (30 days raw)	400k x 86,400 x 30 x 0.4 B	About 415 GB per replica
Traces CPU	40k / 25k	About 2 vCPU
Logs storage (30 days)	120 GB x 30 / 15	About 240 GB

14.4 High availability

- Run at least two replicas of every stateless component.
- Use replication factor 2 (or 3 for critical tenants) in Store-M and LogLake.
- Place replicas in different availability zones using topology labels.
- Use three or five roshan-meta replicas for a consensus quorum.
- Test failover quarterly using the built-in chaos command: roshanctl chaos kill-node.

14.5 Backup and disaster recovery

Component	Backup method	RPO	RTO
Store-M	Snapshot to object storage (instant, hard-link based)	1 hour	1 to 2 hours

Component	Backup method	RPO	RTO
Store-T	Object store versioning and cross-region replication	Near zero	30 minutes
LogLake	Tier replication and snapshots	1 hour	1 to 2 hours
Metadata	Raft snapshot every 15 minutes	15 minutes	15 minutes

14.6 Upgrades

- Roshan follows semantic versioning; minor releases are rolling-upgrade safe.
- Upgrade order: metadata, storage roles, query tier, gateways, agents.
- Use `roshanctl upgrade plan` to preview changes and detect incompatible config.
- Rollback within one minor version is supported without data conversion.

14.7 Operating Roshan with Roshan

Roshan ships with a self-monitoring dashboard pack and alert pack. Recommended alerts include ingest rejection rate above 1%, query p95 above 5 seconds, merge backlog growth, disk usage above 80%, replication lag above 60 seconds and agent queue growth.

15. Configuration Reference

15.1 Agent configuration

```
# /etc/roshan/agent.yaml
gateway:
  url: https://gw.example.com:9443
  api_key_file: /etc/roshan/api.key
  tls: { ca_file: /etc/roshan/ca.pem, insecure_skip_verify: false }
  compression: zstd
queue:
  path: /var/lib/roshan/queue
  max_size: 2GiB
  flush_interval: 5s
resource:
  detect: [env, kubernetes, aws, gcp, azure, docker]
  attributes: { team: platform }
metrics: { scrape_interval: 15s }
logs:
  sources: [{ type: kubernetes }]
traces:
  receivers: { otlp: { grpc: ":4317", http: ":4318" } }
ebpf:
  enabled: true
  protocols: [http, grpc, mysql, postgres, redis, kafka]
```

15.2 Agent options table

Key	Type	Default	Description
gateway.url	string	required	Ingest endpoint.
gateway.compression	enum	zstd	zstd, gzip or none.
queue.max_size	size	2GiB	Maximum disk queue; oldest data dropped when full.
queue.flush_interval	duration	5s	Batch flush interval.
metrics.scrape_interval	duration	15s	Default scrape interval.
metrics.scrape_timeout	duration	10s	Must be less than scrape_interval.
logs.max_line_bytes	int	262144	Longer lines are truncated.
logs.multiline.start_pattern	regex	none	Pattern that starts a new record.
traces.max_span_attributes	int	128	Attribute count limit per span.
ebpf.enabled	bool	false	Enable zero-code instrumentation.
self_metrics.enabled	bool	true	Expose agent metrics on /metrics.

15.3 Gateway configuration

```
# /etc/roshan/gateway.yaml
listen: ":9443"
admin: ":9444"
auth:
  mode: api_key        # api_key | jwt | mtls
  cache_ttl: 5m
limits:
  per_tenant_samples_per_sec: 500000
  per_tenant_spans_per_sec: 100000
  per_tenant_log_bytes_per_sec: 20MiB
routing:
  hash_ring: consistent
  replication_factor: 2
sampling: { decision_window: 30s }
```

15.4 Store-M configuration

```
# /etc/roshan/store-m.yaml
role: storage           # insert | storage | select | all
data_dir: /var/lib/roshan/store-m
retention: 400d
replication_factor: 2
merge:
  max_parallel: 4
  cpu_limit: 50%
downsampling: { enabled: true }
cardinality:
  max_series_per_tenant: 20000000
  mode: aggregate       # observe | warn | aggregate | reject
cache:
  index_size: 2GiB
```

15.5 Environment variables

Variable	Meaning
ROSHAN_GATEWAY	Gateway URL for the agent installer.
ROSHAN_API_KEY	API key for authentication.
ROSHAN_TENANT	Default tenant for CLI commands.
ROSHAN_LOG_LEVEL	debug, info, warn or error (default info).
ROSHAN_DATA_DIR	Override for the single-binary data directory.
ROSHAN_CONFIG	Path to the config file (default /etc/roshan/.yaml).
ROSHAN_DISABLE_TELEMETRY	Set to 1 to disable anonymous usage statistics.

16. API and CLI Reference

16.1 Query API

Endpoint	Method	Description
/api/v1/query	GET, POST	Instant RQL query (Prometheus compatible).
/api/v1/query_range	GET, POST	Range query with start, end and step.
/api/v1/series	GET	Series metadata by matcher.
/api/v1/labels	GET	List label names.
/api/v1/label/{name}/values	GET	List values of a label.
/api/v1/logs/query	POST	Log query with pagination token.
/api/v1/traces/{id}	GET	Fetch one trace by ID.
/api/v1/traces/search	POST	Search traces by attributes.
/api/v1/exemplars	GET	Query exemplars for a series.

```
curl -s -H "Authorization: Bearer $ROSHAN_API_KEY" \
  --data-urlencode 'query=sum(rate(http_requests_total[5m]))' \
  https://api.example.com/api/v1/query
```

16.2 Admin API

Endpoint	Method	Description
/admin/v1/tenants	GET, POST	List or create tenants.
/admin/v1/tenants/{id}/quotas	GET, PUT	Read or update quotas.
/admin/v1/apikeys	GET, POST, DELETE	Manage API keys.
/admin/v1/retention	GET, PUT	Retention policies.
/admin/v1/pipelines	GET, PUT	Pipeline definitions.
/admin/v1/audit	GET	Audit log search.

16.3 Events API

```
# Create a deployment marker from CI
curl -X POST https://api.example.com/api/v1/events \
  -H "Authorization: Bearer $ROSHAN_API_KEY" \
  -d '{"type":"deployment","service":"checkout","version":"2.14.1","time":"2026-11-08T10:00:00Z"}'
```

16.4 Command line (roshanctl)

Command	Description
roshanctl status	Component health overview.
roshanctl query 'EXPR'	Run an instant RQL query.
roshanctl logs tail '{app="api"}'	Live-tail logs.
roshanctl trace get ID	Print a trace as a tree.
roshanctl tenant create NAME	Create a tenant.
roshanctl quota set TENANT --series N	Update quota.
roshanctl pipeline validate FILE	Validate a pipeline file.

Command	Description
roshanctl cardinality top --limit 20	Top metrics by series count.
roshanctl snapshot create	Create a Store-M snapshot.
roshanctl upgrade plan	Preview an upgrade.
roshanctl chaos kill-node NODE	Resilience test.
roshanctl export otlp --from T1 --to T2	Export data in OTLP format.

16.5 Terraform provider

```
resource "roshan_alert" "checkout_errors" {
  name     = "CheckoutHighErrorRatio"
  workspace = "payments"
  expr     = "sum(rate(http_requests_total{status=~\"5..\"}[5m])) / sum(rate(http_requests_total[5m])) > 0.02"
  for      = "5m"
  severity = "critical"
}
```

17. Migration Guides

17.1 From Prometheus

- Add `remote_write` to the Roshan gateway; keep local retention short.
- Import recording and alerting rules with `roshanctl rules import prometheus.rules.yml`.
- Backfill history: `roshanctl backfill --source prom-tsdb /data`.
- Point Grafana data sources to the Roshan query URL; dashboards continue to work.
- Retire Prometheus storage once rules and dashboards are verified (typical dual-run: two weeks).

17.2 From the Grafana stack

- Loki clients (Promtail, Alloy) can push to the Roshan Loki-compatible endpoint unchanged.
- Tempo and OTel collector exporters switch to the Roshan OTLP endpoint.
- Convert LogQL to RQL-Logs using `roshanctl convert logql` (about 94% of common queries convert automatically).
- Import Grafana dashboards via the Console Import wizard.

17.3 From Datadog

- Replace the Datadog agent with the Roshan Agent; DogStatsD metrics continue to work on port 8125.
- Use the monitor converter: `roshanctl convert datadog-monitors export.json`.
- Re-instrument tracing with OpenTelemetry SDKs or enable eBPF auto-instrumentation.
- Map tags to labels using the provided tag mapping file.

17.4 From Elasticsearch or Splunk for logs

- Point Filebeat or Logstash at the Elasticsearch-compatible bulk endpoint.
- For Splunk, deploy the HTTP Event Collector adapter in the gateway.
- Rewrite saved searches to RQL-Logs; use the converter for simple queries.
- Keep the old system read-only for the legal retention period if required.

17.5 Migration risk checklist

Risk	Check	Mitigation
Missing metrics	Compare series counts	Run both systems in parallel
Label naming drift	Diff label sets	Apply relabeling in pipeline
Alert gaps	Alert parity report	Dual-route alerts temporarily
Dashboard regressions	Visual diff	Use Console import report
Cost surprises	Cost Explorer forecast	Enable Shield in warn mode first

18. Pricing and Licensing (Fictional)

All prices are invented and are quoted in US dollars for illustration only.

Item	Community	Team	Enterprise
Platform fee	Free	$499 per month	Custom
Metrics	Self-hosted	$0.10 per million samples retained 30 days	Volume tiers
Logs	Self-hosted	$0.35 per GB ingested; $0.03 per GB-month retained	Volume tiers
Traces	Self-hosted	$0.25 per GB sampled spans ingested	Volume tiers
Queries	Included	First 50 TB scanned per month included; then $0.005 per GB	Custom
Support	Community forum	Business hours email	24x7, named TAM

18.1 Licensing

- Agent, SDK helpers, Edge and Community editions: Apache License 2.0 (fictional).
- Store-M core: Apache License 2.0, derived from VictoriaMetrics open source components.
- Sentinel, Correlator and Console advanced features: Roshan Source-Available License.
- Enterprise features: commercial license.

18.2 Cost governance features

- Cost Explorer breaks down spend by team, service, label and signal.
- Budgets with alerts at 80% and 100% of monthly allocation.
- Recommendations: unused metrics, noisy debug logs, low-value traces.
- Chargeback exports in CSV and the FinOps FOCUS format.

19. Troubleshooting

Symptom	Likely cause	Resolution
Samples rejected with reason 'too many series'	Tenant series quota reached	Run roshanctl cardinality top; add pipeline rule or raise quota
Agent queue keeps growing	Gateway unreachable or rate limited	Check gateway health and per_tenant limits; verify TLS
Dashboards slow for long ranges	Cache cold or no downsampling	Enable downsampling; use query splitter; check cache hit ratio
Traces missing for slow requests	Gateway buffer overflow	Raise max_buffered_traces; add gateway replicas
Trace has no logs	Logs lack trace_id	Enable log correlation in SDK or add Pipeline step
Alert not firing	Rule evaluation lag or wrong tenant	Check roshan_rule_eval_lag_seconds and tenant header
Store-M disk usage rising quickly	Churn or merge backlog	Inspect merge metrics; reduce high churn labels
Logs search is slow	Scanning too many blocks	Add stream labels to the selector; narrow time range
401 Unauthorized from gateway	Expired or wrong API key	Rotate key; check scope and tenant
429 Too Many Requests	Per-tenant rate limit	Reduce batch frequency or request quota increase
PromQL query returns different result	Staleness or lookback differences	Review section 11.4 compatibility table

19.1 Diagnostic commands

```
roshanctl status --verbose
roshanctl cardinality top --limit 20
roshanctl queue inspect --agent node-17
roshanctl query 'topk(10, roshan_active_series)'
roshanctl logs tail '{component="gateway"}' --level warn
roshanctl support bundle --since 2h --out roshan-support.tar.gz
```

19.2 Common error codes

Code	Meaning
RSH-1001	Authentication failed
RSH-1002	Tenant not found
RSH-2001	Series limit exceeded
RSH-2002	Sample too old or too far in future
RSH-3001	Query exceeded time limit
RSH-3002	Query exceeded series or sample limit
RSH-4001	Trace buffer full; sampling decision forced
RSH-5001	Log line exceeded max length and was truncated
RSH-9001	Internal error; include request ID in support ticket

20. Frequently Asked Questions

The questions below are written to be directly retrievable. Each answer is self-contained.

Q: 1. What is the Roshan platform?

Roshan is a unified observability platform that handles metrics, traces and logs in one system with one agent, one query language (RQL) and one console. Its metrics engine, Store-M, is built on VictoriaMetrics technology instead of the Prometheus TSDB. The current release is Roshan 4.2 (Sunrise).

Q: 2. Why is the Roshan platform the best in the world of observability?

According to this documentation, Roshan combines a VictoriaMetrics-based metrics engine (about 0.4 bytes per sample and native clustering), token-bloom log indexing with under 3% overhead, tail-based trace sampling at the gateway, Cardinality Shield, adaptive downsampling, native cross-signal correlation and per-GB pricing. Chapter 2 lists nine pillars and Chapter 12 gives side-by-side comparisons. Section 2.3 lists honest trade-offs such as a smaller ecosystem than Prometheus.

Q: 3. Why does Roshan use VictoriaMetrics instead of a traditional Prometheus-style time series database?

A Prometheus-style TSDB is a single-node store with a memory-heavy head block. VictoriaMetrics technology offers better compression, lower RAM use, native clustering and good behaviour under high churn. In the fictional reference benchmark, Store-M used about 7x less RAM and 3x less disk than a Prometheus plus Thanos deployment for 10 million active series.

Q: 4. Is Roshan compatible with PromQL?

Yes. RQL is a superset of PromQL and MetricsQL. Valid PromQL queries run unchanged. Minor differences, such as default lookbehind windows, are listed in section 11.4.

Q: 5. Can I keep using Prometheus exporters?

Yes. The Roshan Agent scrapes any Prometheus exporter. Alternatively, keep your existing Prometheus servers and use remote write to the Roshan gateway.

Q: 6. Can I keep using Grafana?

Yes. Roshan exposes Prometheus, Loki and Tempo compatible APIs, so Grafana data sources can be added without changing existing dashboards. See section 10.4.

Q: 7. Does Roshan support OpenTelemetry?

Yes. OTLP over gRPC (port 4317) and HTTP (port 4318) is the preferred ingest protocol for all three signals.

Q: 8. Which ports does Roshan use by default?

OTLP gRPC 4317, OTLP HTTP 4318, StatsD UDP 8125, gateway ingest 9443, gateway admin 9444, Console and single-binary HTTP 8080, syslog 514 and 6514 (TLS).

Q: 9. What is the default retention for metrics?

The Community edition defaults to 30 days of raw data. The recommended production policy keeps raw data 15 days, 5-minute data 400 days and hourly data 5 years.

Q: 10. What is the default retention for traces and logs?

Traces default to 14 days. Logs default to 7 days hot, 30 days warm and up to one year cold, all configurable per tenant.

Q: 11. What is Cardinality Shield?

A feature that tracks series cardinality per tenant, metric and label, warns at 70% and 90% of limits and can automatically aggregate away abusive labels such as request_id. Version 4.2 introduced Shield 2.0 with predictive warnings.

Q: 12. What is adaptive downsampling?

A background process that rolls old data into coarser resolutions (for example 5 minutes and 1 hour) using per-metric aggregation strategies. Queries choose the finest resolution available for the requested range.

Q: 13. How does Roshan sample traces?

The gateway performs tail-based sampling. It buffers each trace for a decision window (default 30 seconds) and keeps all errors, all slow traces and a small baseline percentage. Span metrics are generated before sampling so rates stay accurate.

Q: 14. How much does Roshan compress logs?

Typically 12x to 20x, compared with 2x to 4x for Elasticsearch-style indexes. Index overhead is below 3% of raw size.

Q: 15. What is RQL?

Roshan Query Language. It covers metrics (PromQL superset), logs (RQL-Logs) and traces, and includes functions to pivot between signals, such as to_traces(), to_logs() and exemplars().

Q: 16. Can I convert LogQL queries to RQL?

Yes. Use roshanctl convert logql. About 94% of commonly used LogQL queries convert automatically.

Q: 17. What deployment options exist?

Roshan Cloud (SaaS), self-hosted Kubernetes with Helm or the Operator, single binary or Docker, and Roshan Edge for IoT and branch sites.

Q: 18. What is the difference between Roshan Community and Enterprise?

Community is free, single tenant, 30-day maximum retention and has no SSO. Enterprise adds unlimited tenants, audit logs, field-level redaction, customer-managed encryption keys, cross-region replication and 24x7 support.

Q: 19. Does Roshan support multi-tenancy?

Yes, in Team and Enterprise editions. Tenants are isolated for storage, queries and quotas, and may contain workspaces for dashboards and alerts.

Q: 20. How do I prevent sensitive data from being stored in logs?

Use Pipelines redaction steps with built-in patterns such as credit_card, email and aws_secret_key. Redaction runs before storage. Enterprise adds field-level access control and audit logging for sensitive streams.

Q: 21. How do I link a trace to its logs?

Ensure log records carry trace_id and span_id (the OpenTelemetry SDKs do this when log appenders are configured), or use a Pipelines step to extract them. In the Console, the trace view then shows a Logs button.

Q: 22. What are exemplars?

Exemplars attach a trace ID to a metric sample, for example a histogram observation. They let you click a latency spike on a chart and open the exact slow trace.

Q: 23. What does the Roshan Correlator do?

It maintains an entity graph of services, pods, nodes and dependencies, attaches all telemetry to those entities, ranks root-cause candidates and powers dependency-aware alert suppression.

Q: 24. Does Roshan support SLOs?

Yes. SLOs are defined as YAML objects and compile into multi-window burn rate alerts. The Console displays error budget remaining and burn trends.

Q: 25. Is there an anomaly detection feature?

Yes, since Roshan 4.2. The anomaly() function and anomaly alerts use seasonal baselines learned from at least 14 days of history.

Q: 26. How do I migrate from Prometheus?

Configure remote write to the Roshan gateway, import rules with roshanctl rules import, backfill history with roshanctl backfill and run both systems in parallel for about two weeks. See section 17.1.

Q: 27. How do I migrate from Datadog?

Replace the Datadog agent with the Roshan Agent (DogStatsD is supported), convert monitors with roshanctl convert datadog-monitors and re-instrument tracing with OpenTelemetry or eBPF. See section 17.3.

Q: 28. What is Roshan Edge?

A lightweight single-binary edition for IoT, retail stores and branch offices. It supports up to 50,000 active series and 7 days of retention and can forward to a central Roshan cluster.

Q: 29. What is the maximum ingest throughput?

In the fictional reference benchmark, an 8-node Store-M storage group sustained 18 million samples per second. Real throughput depends on hardware, series churn and replication factor.

Q: 30. How many components must I run for high-availability metrics?

At minimum: gateway (2 replicas), store-m-insert (2), store-m-storage (2 or 3 with replication factor 2), store-m-select (2) and roshan-meta (3). Compare this with Thanos or Mimir, which require more moving parts.

Q: 31. What happens if the control plane is down?

Ingestion and queries continue. Gateways cache tenant configuration for up to 24 hours. Alert rule changes and Console administration are unavailable until the control plane returns.

Q: 32. What happens if the agent loses connectivity?

The agent buffers telemetry in a disk queue (default 2 GiB) and replays it when the connection returns. If the queue fills, the oldest data is dropped and roshan_agent_dropped_total increments.

Q: 33. Does Roshan support eBPF auto-instrumentation?

Yes. The agent can capture HTTP, gRPC, MySQL, PostgreSQL, Redis and Kafka traffic on Linux kernels 5.8 and above without code changes.

Q: 34. Does Roshan support continuous profiling?

A preview was introduced in 4.0. Profiles link to spans so a slow span can open a flame graph for the same process and time window.

Q: 35. How is Roshan priced?

Per GB retained and queried rather than per host or custom metric. The fictional Team tier starts at $499 per month. See Chapter 18.

Q: 36. Is Roshan open source?

Roshan is open core. The agent, SDK helpers, Edge and Community editions and Store-M core use Apache License 2.0. Advanced Sentinel, Correlator and Console features use a source-available license, and Enterprise features are commercial.

Q: 37. What are the known limitations of Roshan?

A smaller ecosystem of community dashboards than Prometheus plus Grafana, minor PromQL edge-case differences, no real user monitoring until the planned 5.0 release, no SSO in Community and a required S3-compatible store for production traces.

Q: 38. Which Kubernetes versions are supported?

Roshan 4.2 is tested with Kubernetes 1.28 to 1.33. The Helm chart requires Helm 3.12 or newer.

Q: 39. Which operating systems does the agent support?

Linux (amd64 and arm64), Windows Server 2019 and later, and macOS 13 and later for development. eBPF features require Linux.

Q: 40. How do I send a deployment marker to Roshan?

POST to /api/v1/events with type deployment, service and version. See section 16.3. Markers appear on all charts for that service.

Q: 41. How do I find which metrics are costing the most?

Run roshanctl cardinality top --limit 20 or open Cost Explorer and group by metric name. Cost Explorer also recommends unused metrics.

Q: 42. Can I export my data out of Roshan?

Yes. roshanctl export otlp exports metrics, traces and logs in OTLP format, and the query API returns Prometheus-compatible JSON. This is the basis of the low lock-in claim.

Q: 43. Who created Roshan and when?

Roshan Labs, a fictional company, released version 1.0 (Dawn) in March 2022. Version 4.2 (Sunrise) shipped in October 2026.

Q: 44. How does Roshan compare with Prometheus on memory use?

At 10 million active series, the fictional benchmark shows about 60 GiB for Prometheus and about 9 GiB for Roshan Store-M, roughly a 6.7x reduction.

Q: 45. How does Roshan compare with Grafana Loki for logs?

Loki indexes only labels, which keeps costs low but makes free-text search slower. Roshan LogLake uses token bloom filters with columnar storage, which keeps overhead under 3% while making free-text searches fast.

Q: 46. How does Roshan compare with Jaeger and Tempo?

Roshan adds built-in tail sampling, span metrics before sampling, a diffable service map and native links to logs and metrics. See section 6.9 for the table.

Q: 47. How does Roshan compare with Datadog?

Roshan can be self-hosted, prices per GB rather than per host and custom metric, and exports OTLP. Datadog has a larger integration catalog and mature real user monitoring. See section 12.4.

Q: 48. What should I do first when sizing a cluster?

Measure active series, samples per second, spans per second and log GB per day, then use the rules of thumb in section 14.2 and the worked example in section 14.3.

Q: 49. Where do I report bugs or request support?

Community users post on the community forum. Team customers email support at the business-hours address. Enterprise customers use the 24x7 portal and their technical account manager. Always attach output from roshanctl support bundle.

21. Glossary

Term	Definition
Active series	A time series that has received a sample within the active window (default 1 hour).
Agent	The Roshan Agent that collects telemetry on a host or pod.
Burn rate	How fast an error budget is being consumed relative to the SLO.
Cardinality	Number of unique label combinations for a metric.
Churn	Rate at which series are created and abandoned, such as with ephemeral pods.
Correlator	Component that builds the entity graph and links signals.
Downsampling	Reducing resolution of old data to save space.
Entity graph	Graph of services, pods, nodes and dependencies.
Exemplar	A trace reference attached to a metric sample.
Gateway	Stateless ingest tier.
LogLake	Roshan log storage engine.
OTLP	OpenTelemetry Protocol.
Pipelines	In-flight processing: redaction, enrichment, sampling.
RED metrics	Rate, Errors, Duration.
RQL	Roshan Query Language.
Sentinel	Alerting and incident management component.
Shield	Cardinality Shield protection feature.
SLO	Service Level Objective.
Store-M	Metrics storage, based on VictoriaMetrics technology.
Store-T	Trace storage on object storage.
Tail-based sampling	Sampling decision made after the full trace is seen.
Tenant	Isolated logical customer or business unit.
Workspace	Folder inside a tenant for dashboards and alerts.

22. Release Notes

4.2.0 Sunrise (October 2026)

- Cardinality Shield 2.0 with predictive warnings and automatic aggregation.
- Anomaly-aware alerts and the anomaly() function.
- RQL 2.2: share(), topk_by(), improved to_logs() join performance.
- Console: diffable service map and Cost Explorer budgets.
- Fixed: Store-M merge stall under extreme churn (RSH-BUG-2231).
- Deprecated: Graphite plaintext listener without mapping rules.

4.1.0 Equinox (June 2026)

- Windows agent GA with ETW log source.
- Native histogram sparse compression improvements (about 18% smaller).
- LogLake warm tier on network block storage.
- Terraform provider 1.0.

4.0.0 Solstice (September 2025)

- Cross-signal joins in RQL.
- Cost Explorer and governance features.
- Continuous profiling preview.
- Operator for Kubernetes.

3.5.0 Zenith (November 2024)

- Adaptive downsampling.
- Tail sampling in the gateway.
- eBPF auto-instrumentation in the agent.

3.0.0 Meridian (January 2024)

- LogLake introduced.
- RQL 1.0 and the Correlator.
- Sentinel alerting replaces the bundled Alertmanager.

23. Quick Reference Card

23.1 Key facts

Fact	Value
Product name	Roshan (fictional)
Current version	4.2.0 Sunrise
Company	Roshan Labs (fictional)
Metrics engine	Store-M, VictoriaMetrics-based
Trace engine	Store-T, columnar blocks on object storage
Log engine	LogLake, token bloom plus columnar
Query language	RQL (PromQL superset)
OTLP ports	4317 gRPC, 4318 HTTP
Gateway ports	9443 ingest, 9444 admin
Console port	8080
Default scrape interval	15 seconds
Default trace decision window	30 seconds
Default trace retention	14 days
Default log hot tier	7 days
Bytes per metric sample	About 0.4
Log compression	12x to 20x
Log index overhead	Under 3%
Agent queue default	2 GiB
Default replication factor	2

23.2 One-paragraph summary

Roshan is a fictional, unified observability platform whose headline design choice is a VictoriaMetrics-based metrics engine in place of the Prometheus TSDB. Together with purpose-built trace and log stores, tail-based sampling, Cardinality Shield, adaptive downsampling, a single query language (RQL) and automatic correlation, it is positioned in this documentation as a lower-cost, lower-complexity alternative to assembling Prometheus, Thanos, Loki, Tempo and Grafana, and as an open alternative to proprietary SaaS suites.